

Egypt Real Estate Appraiser

Machine Learning-Based Property Price Prediction

Dubizzle Scrape Dataset | 74,240 Records | $R^2 = 0.7576$ | MAPE = 27.87%

Team Members:

Ahmed El-Shawadify
Haitham Hany Ahmed
Hayam Abdullah
Sara Mohamed

1. Project Proposal

1.1 Objective

Help end users get a close estimation of real estate prices in Egypt. This should prove very beneficial to both real-estate seekers (especially from outside the domain) and investors alike.

To achieve this goal, we leverage Python's machine learning libraries to analyse a collected dataset and build a reliable price prediction model for the Egyptian property market.

1.2 Project Description

This project involves collecting and processing Egyptian real estate listing data from Dubizzle.com.eg. We use Python to perform data cleaning, statistical analysis, and predictive modelling to estimate property values based on key property features, including location, size, property type, ownership status, payment option, and completion status.

1.3 Tools & Design

System Architecture: A modular Python pipeline — Data Collection, Preprocessing, Model Training, Evaluation, and Web Deployment.

Development Environment: Jupyter Notebooks for EDA and VS Code for script development.

Data Collection: Two-stage web scraping pipeline from Dubizzle.com.eg (Stage 1: URL collection; Stage 2: property detail extraction).

Data Management: Pandas DataFrames for manipulation during training and inference.

User Interface: Interactive Streamlit dashboard for real-time property price estimation.

1.4 Technologies

Python 3.11

Pandas & NumPy

Scikit-learn

Matplotlib & Seaborn

Selenium + BeautifulSoup (Web Scraping)

Jupyter Notebooks & Streamlit

Git / GitHub

1.5 Team Members & Responsibilities

Phase	Main Tasks	Contributors
Phase 1: Data Collection & Web Scraping	- Building Data Collection Pipeline - Two-stage Web Scraping Implementation - Handling Pagination, Blocking & Dynamic Content - Raw Data Storage	Haitham Hany Ahmed (Lead) Ahmed El-Shawadify Hayam Abdullah Sara Mohamed
Phase 2: Data Cleaning, Feature Engineering & EDA	- Data Cleaning & Preprocessing - Feature Engineering - Exploratory Data Analysis (EDA) - Data Visualization - Final Dataset Preparation	Ahmed El-Shawadify (Lead) Haitham Hany Ahmed Hayam Abdullah Sara Mohamed
Phase 3: ML Modeling, Evaluation & Deployment	- ML Model Training (Multiple Models) - Model Evaluation & Hyperparameter Tuning - Model Selection - Streamlit App Development - Application Deployment	Sara Mohamed (Lead) Ahmed El-Shawadify Haitham Hany Ahmed Hayam Abdullah
Phase 4: Documentation, Testing & Presentation	- Full Project Documentation - Unit & Integration Testing - Presentation Slides & Demo - Professional README & Final Report - Project Organization & Cleaning	All Team Members Hayam Abdullah (Lead)

1.6 Project Plan

Week 1 — Setup & Data Collection:

- Build Stage 1 scraper: collect listing URLs from Dubizzle by category and city.
- Build Stage 2 scraper: visit each URL and extract full property details.
- Set up GitHub repository, Python virtual environment, and project structure.

Week 2 — Data Cleaning & EDA:

- Clean collected data: fix districts, handle missing values, remove outliers.
- Perform EDA: price distributions, location analysis, new feature impact.
- Engineer features: bounded location pairs, ownership/payment/completion encoding.

Week 3 — Modelling:

- Train stratified models (one per property type): Random Forest vs Gradient Boosting.
- Evaluate on held-out test sets: MAE, RMSE, R^2 , MAPE, CV R^2 .
- Compare and select best model per type.

Week 4 — Deployment & Documentation:

- Complete Streamlit app with cascading Governorate → City → District dropdowns.
- Compile final documentation and DEPI presentation.
- Push all code and model artefacts to GitHub.

2. Literature Review

2.1 Background

Property valuation is a longstanding challenge in real estate. Traditional methods rely on manual comparative market analysis, which is slow, costly, and inconsistent. Machine learning has increasingly been applied to automate valuations, with ensemble methods (Random Forest, Gradient Boosting) consistently outperforming linear models due to the non-linear relationships between features and market price.

Studies in global markets report R^2 values of 0.70–0.95 when rich feature sets are available. Location, area, and property type are universally identified as the most important predictors. The Egyptian market has received limited academic attention, partly due to data scarcity.

2.2 Why the Kaggle Dataset Was Insufficient

The initial project phase used a publicly available Kaggle dataset scraped from PropertyFinder.eg. Critical limitations were identified:

- Scale: Only 16,083 records — insufficient to represent Egypt's diverse property market.
- Location: Only Governorate and a region field, often region == governorate (no signal).
- Missing features: No Ownership, Completion Status, Payment Option, or Compound name.
- Static: Could not be refreshed as market conditions evolved.
- Performance: Best $R^2 = 0.4919$, MAPE = 56.9% — insufficient for practical use.

Dataset	Source	Records	R^2	MAPE	MAE
Kaggle	PropertyFinder scrape	16,083	0.4919	56.9%	4.55M
data.xlsx	PropertyFinder curated	81,076	0.6873	33.0%	4.17M
Dubizzle (Final)	Dubizzle scrape	74,240	0.7576	27.87%	3.53M

Table 1 — Dataset evolution across three iterations.

2.3 How Scraping Improved Accuracy

Replacing the Kaggle dataset with a custom Dubizzle scrape produced large improvements:

- R^2 improved: 0.49 → 0.76 (+0.27) — the model now explains 76% of price variance.
- MAPE improved: 57% → 27.87% — prediction error nearly halved.
- Three sources of improvement: (1) New features — Ownership, Completion, Payment; (2) Richer location — City/District with 1,473 combinations vs 303 previously; (3) Scale — 74,240 vs 16,083 records.

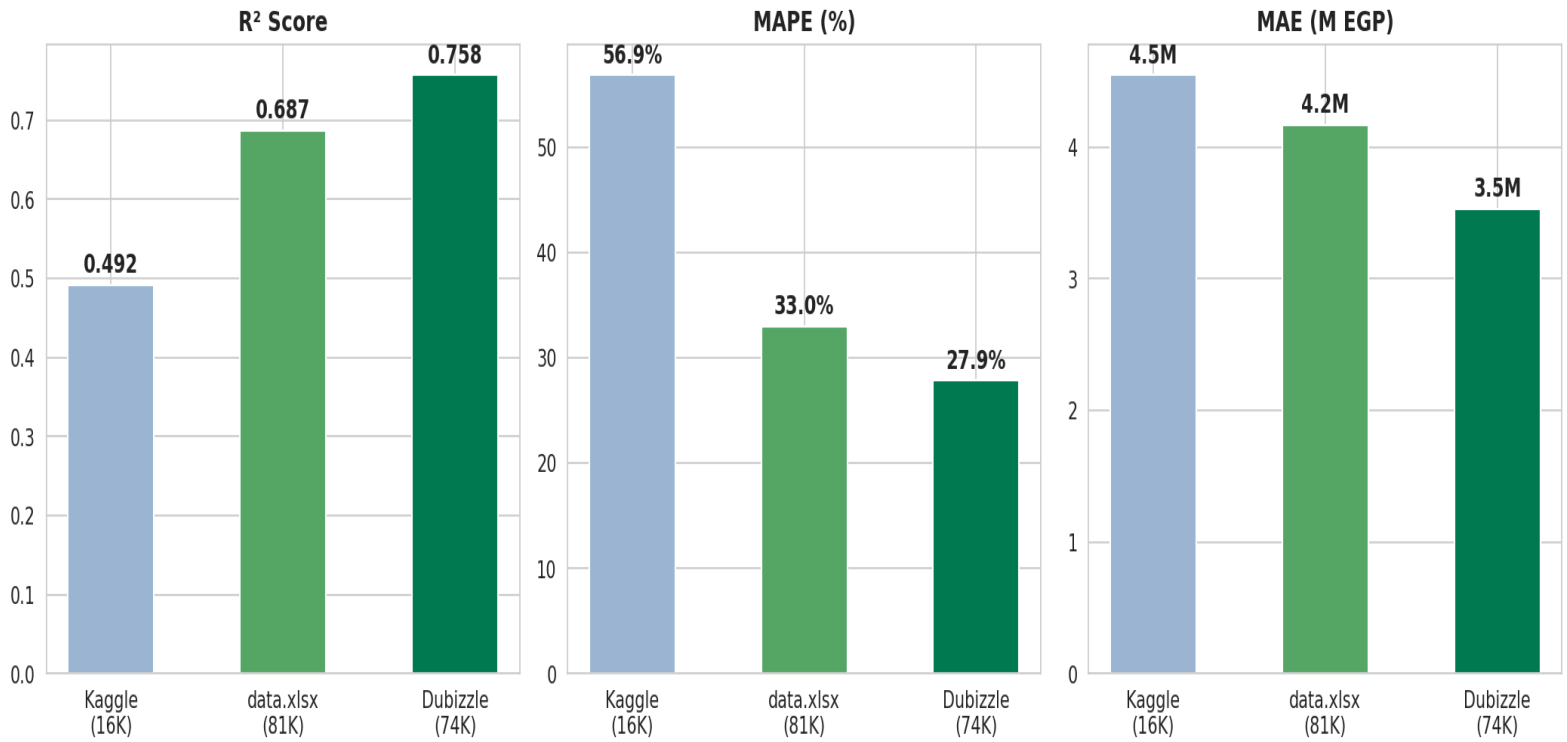


Figure 1 — R^2 , MAPE, and MAE improvement across three dataset versions.

2.4 Related Work Insights

- Tree-based ensemble methods outperform linear regression for non-linear real estate markets.
- Log-transforming right-skewed price distributions consistently reduces MAPE.
- Stratified (per-type) modelling is more domain-logical than unified models.
- Location granularity is the most critical factor: district-level data outperforms governorate-only.

3. Data Collection — Two-Stage Scraping Pipeline

3.1 Source: Dubizzle.com.eg

- Dubizzle Egypt is Egypt's largest property marketplace. Unlike Selenium-based scrapers, the final pipeline uses aiohttp (async HTTP) — no browser required, 40 concurrent workers, orders of magnitude faster.
- robots.txt compliance:
 -  Allowed: /en/properties/{category}/{city}/ — category + city listing pages.
 -  Blocked: /en/properties/ and /en/properties/?product=all.

3.2 Stage 1 — URL Collection

Stage 1 visits category + city listing pages and extracts all listing URLs. Output: URLs.xlsx (107,276 URLs). This stage is run separately and feeds Stage 2.

- Categories: apartments-duplex-for-sale, villas-for-sale, townhouses-for-sale, twin-houses-for-sale, duplexes-for-sale, penthouses-for-sale, vacation-homes-for-sale.
- Pagination: 24 listings per page; navigated via ?page=N parameter.
- Output: URLs.xlsx — one URL per row with city and category labels.

3.3 Stage 2 — dubizzle_scraper.py (aiohttp Async)

The team-authored dubizzle_scraper.py visits each URL from URLs.xlsx and extracts full property details. Key technical features:

```
# Setup (one-time)
pip install aiohttp beautifulsoup4 lxml openpyxl

# Run
python dubizzle_scraper.py                # full run (~107K URLs, 4-6 hours @ 40
workers)
python dubizzle_scraper.py --limit 500    # quick test
python dubizzle_scraper.py --workers 20  # adjust concurrency
python dubizzle_scraper.py --resume      # continue after interruption
```

- Async I/O: aiohttp with configurable concurrency (default 40 workers). No browser, no Selenium.
- Auto-resume: .checkpoint.csv tracks completed URLs — safe to Ctrl+C and restart any time.
- Incremental save: Excel written every 1,000 rows — no data loss on interruption.
- Rate-limit detection: automatic backoff on HTTP 429/403 responses.
- Status tracking: each URL tagged OK, HTTP 404, HTTP 502, TIMEOUT, or BLOCKED.
- Extraction: tries Next.js __NEXT_DATA__ JSON first; falls back to HTML parser (lxml + BS4).

3.4 Scraping Results

- Total URLs collected (Stage 1): 107,276
- Successfully scraped — Status OK (Stage 2): 106,532 (99.3% success rate)
- After cleaning and deduplication: 74,240 final records
- Removed: duplicate listings (same price + area + bedrooms + location), zero prices, extreme outliers
- Final coverage: 8 property types, 18+ governorates, 1,473 city+district combinations

4. Exploratory Data Analysis

4.1 Price Distribution

The distribution is right-skewed (skewness 2.63). Median price is 9.5M EGP; mean is 14.0M EGP. The long tail reflects luxury villas and coastal properties.

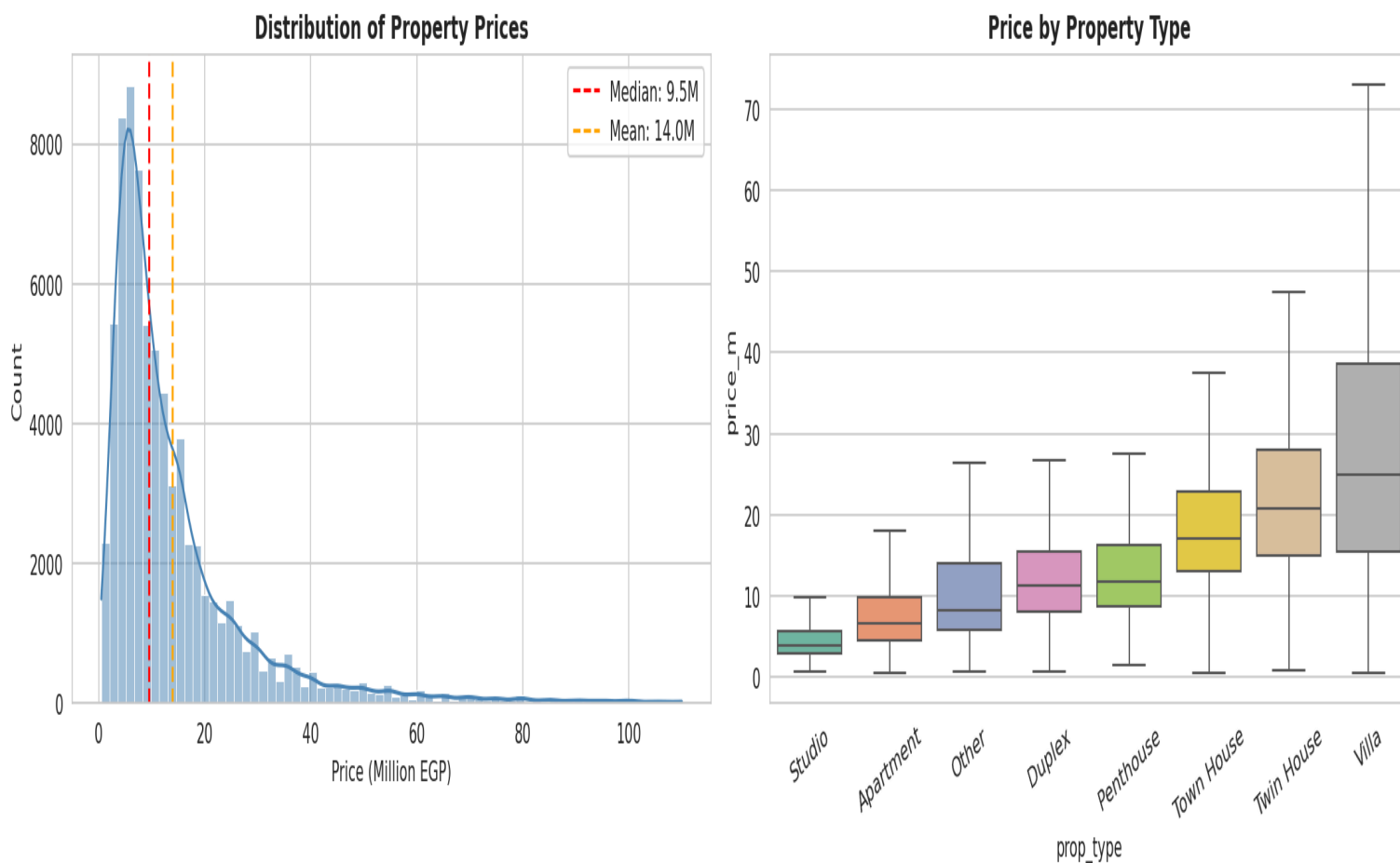


Figure 2 — Price histogram and box-plots by property type.

4.2 Location Analysis

Red Sea and Matruh governorates command the highest median prices (coastal premium). At city level, New Capital City and North Coast lead the premium segment.

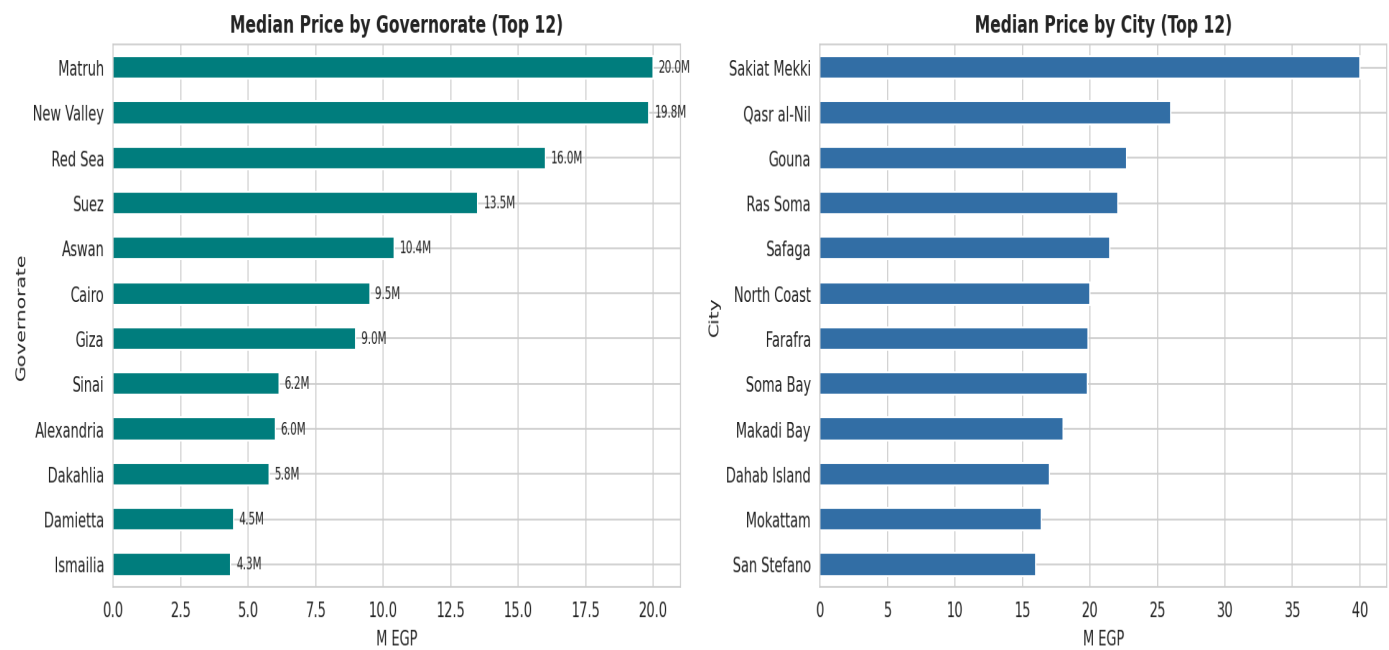


Figure 3 — Median price by governorate and top 12 cities.

4.3 Feature Distributions

Most properties are 80–250 m², with 2–4 bedrooms and 1–3 bathrooms.

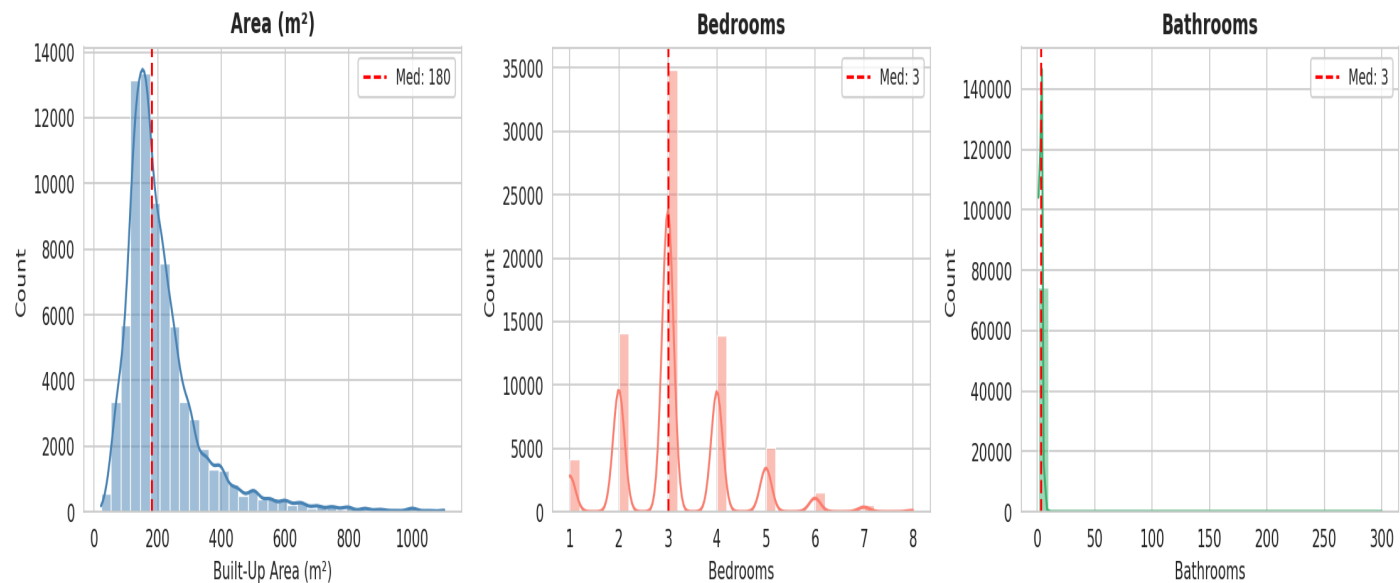


Figure 4 — Distributions of area, bedrooms, and bathrooms.

4.4 New Features: Ownership, Completion & Payment

All three new features show clear price differentiation. Primary+Ready+Cash listings are consistently the most expensive.

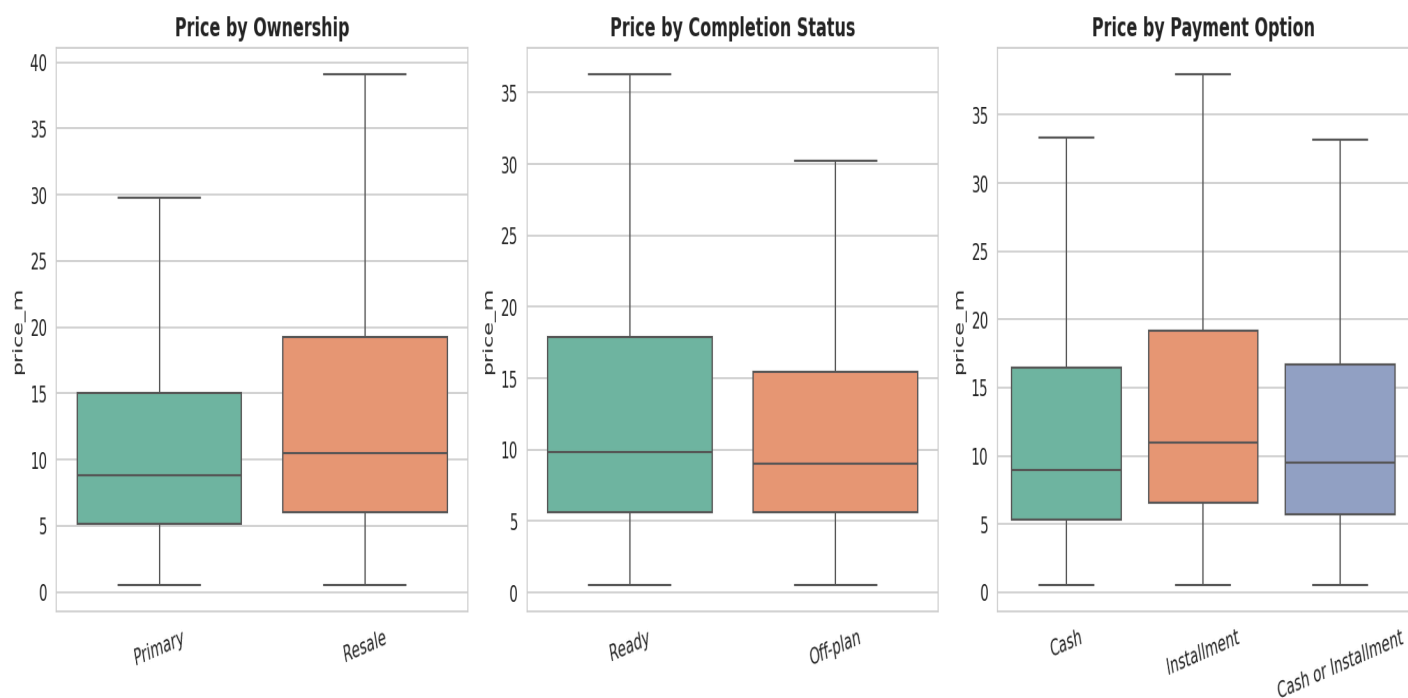


Figure 5 — Price by Ownership, Completion Status, and Payment Option.

4.5 Area vs. Price

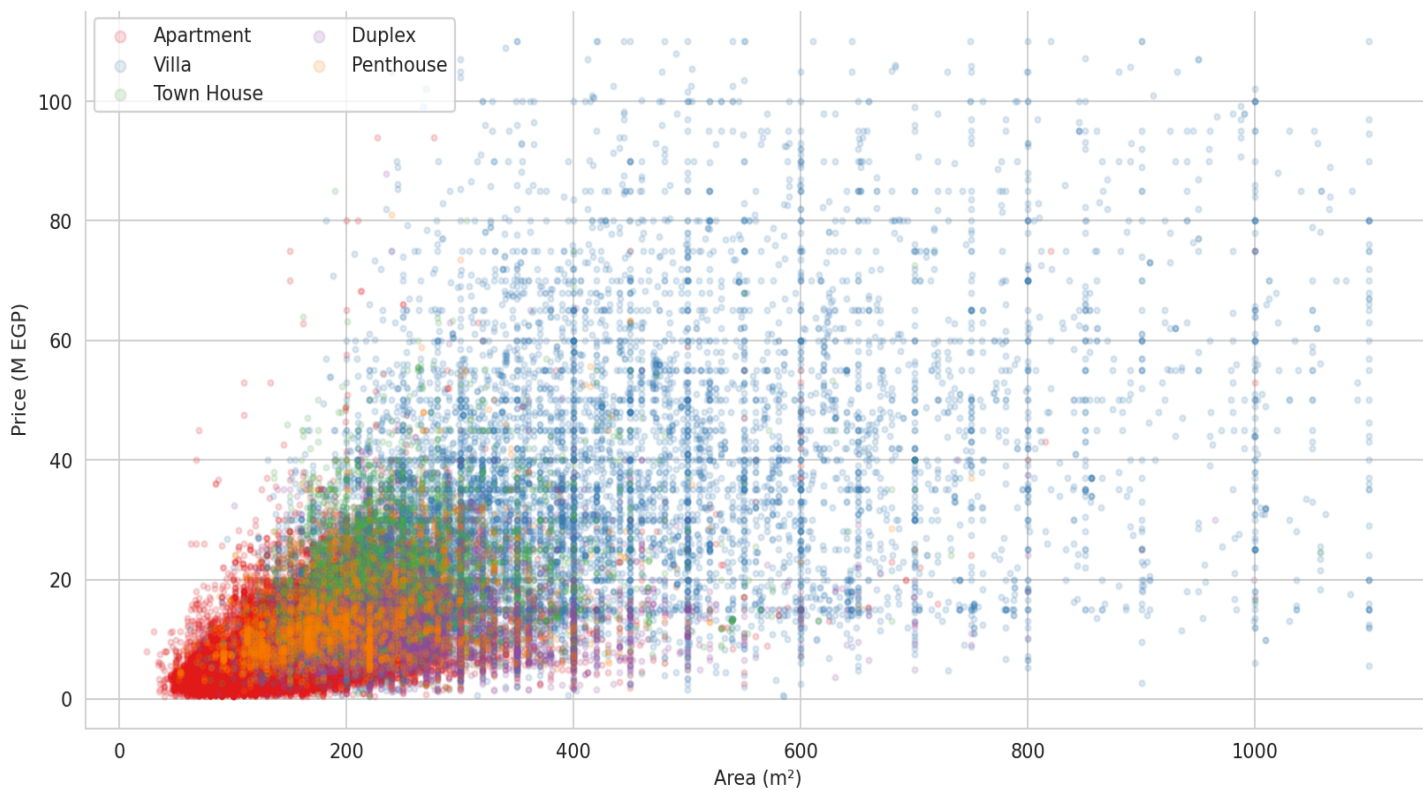


Figure 6 — Area vs. price by type. Non-linear relationship confirms stratified modelling choice.

4.6 Log-Transform

Log-transforming the target variable reduces skewness from 2.63 to 0.08, improving model fit for all algorithms.

Figure 6 — Log-Transform

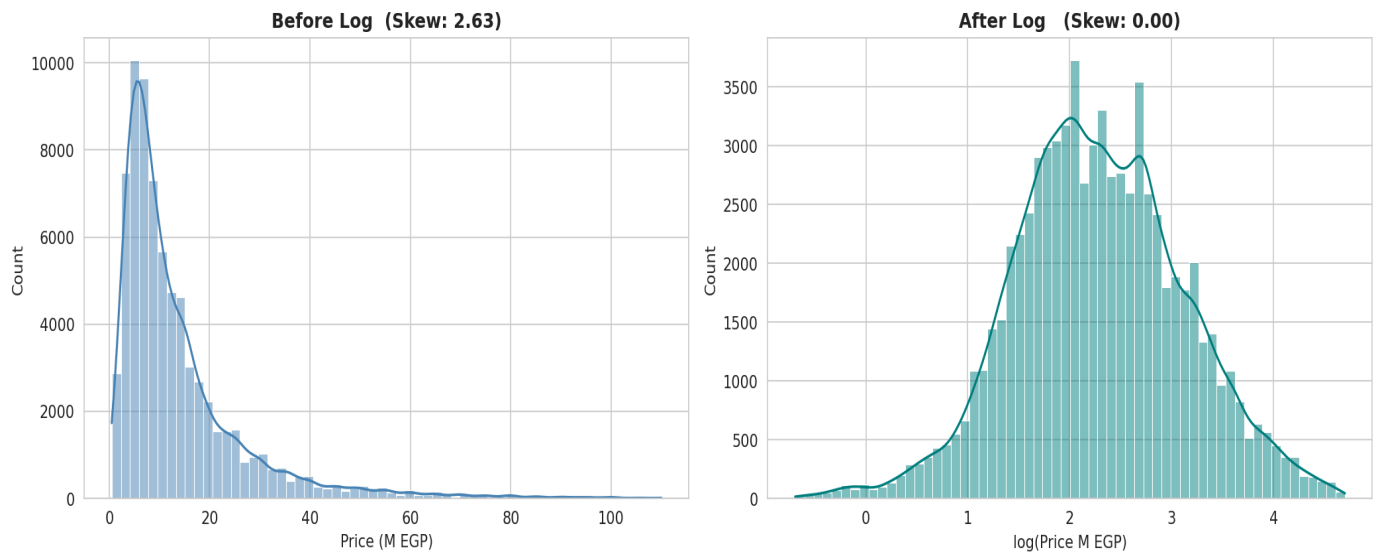


Figure 6 — Price before and after log-transform. Skewness: 2.63 → 0.08.

4.7 Feature Correlations

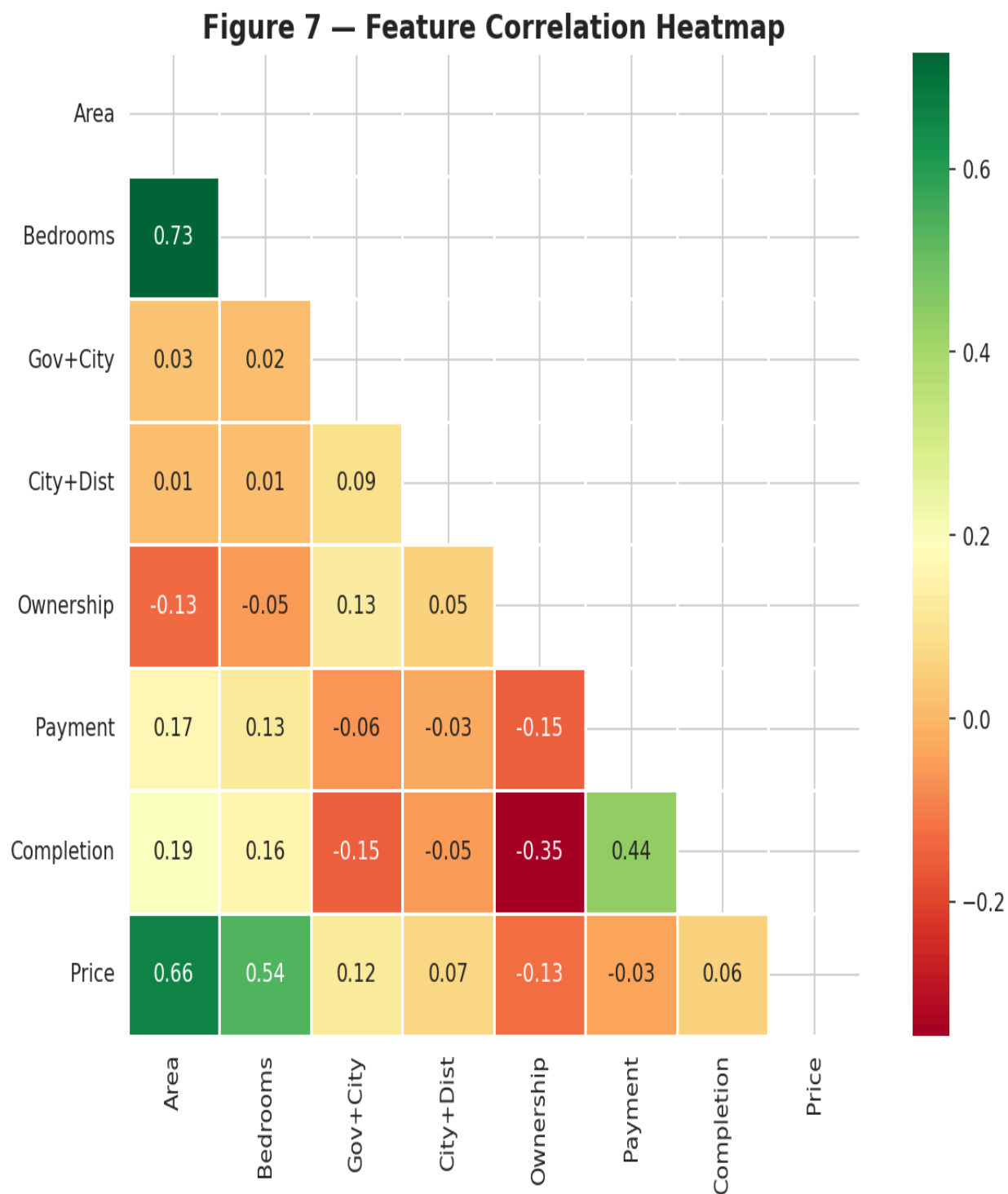


Figure 7 — Correlation heatmap. City+District and Area are the strongest price predictors.

5. Feature Engineering & Data Preparation

Step	Action	Details
Status filter	Kept only valid records	Status = OK → 106,532 of 107,276 retained
Type normalization	Standardized property types	iVilla → Villa; Hotel Apartment + Roof → Other
Price outliers	Retained realistic range	0.5M–99.5th percentile (~110M EGP)
Area outliers	Limited extreme values	20m ² –99.5th percentile (~800m ²)
Bedrooms	Restricted valid range	1–8 bedrooms
Duplicate removal	Removed exact matches	Based on price + area + bedrooms + city + district
District fix	Corrected missing/invalid values	Missing → Compound → City; District==City → Compound if available
Gov+City	Created bounded pair	“Governorate — City” → 181 combinations
City+District	Primary location feature	“City / District” → 1,473 combinations
Ownership encoding	Converted categorical to numeric	Primary=1, Resale=0, Unknown=0.5
Payment encoding	Converted categorical to numeric	Cash=1, Installment=0, Cash or Installment=0.5
Completion encoding	Converted categorical to numeric	Ready=1, Off-plan=0, Unknown=0.5
Log-transform	Target transformation	Training target = $\log(\text{price_m})$; back-transform via $\exp()$
Train/Test split	Dataset partition	80% train / 20% test, random_state=42
Bathrooms removed	Dropped correlated feature	Correlation 0.79 with bedrooms; 1→2 bath jump reflects market tier, not causal effect

6. Machine Learning Models

6.1 Stratified Approach (Per Property Type)

Rather than encoding property type as an arbitrary number, we train a dedicated model for each type. This allows each model to learn the specific area-price and location-price dynamics of its segment. A single unified model would conflate the very different relationships in Apartment pricing vs Villa pricing.

Features used in all type-specific models:

- area_sqm — Built-up area in square metres (strongest numeric predictor).
- bedrooms — Number of bedrooms (bathrooms excluded due to collinearity).
- gc_enc — Gov+City label-encoded (181 classes).
- cd_enc — City+District label-encoded (1,473 classes) — most important feature overall.
- ownership_enc — Primary=1, Resale=0 (significant price premium for Primary).
- payment_enc — Cash=1, Installment=0 (Cash skews premium segment).
- completion_enc — Ready=1, Off-plan=0 (Ready commands immediate availability premium).

6.2 Model Comparison Results

Property Type	Best Model	Records	MAE (M EGP)	R ²	MAPE
Apartment	Random Forest	43,399	2.056	0.5336	28.1%
Villa	Random Forest	13,975	7.933	0.6019	29.6%
Town House	Gradient Boosting	5,565	3.816	0.5611	24.1%
Twin House	Gradient Boosting	3,387	4.512	0.6328	23.2%
Duplex	Gradient Boosting	3,435	3.322	0.4098	31.4%
Penthouse	Gradient Boosting	2,577	3.373	0.5029	23.6%
Studio	Random Forest	994	1.224	0.5882	25.0%
Global	—	74,240	3.532	0.7576	27.87%

6.3 Model Descriptions

Random Forest (Ensemble Bagging)

200 decision trees on bootstrapped subsets. Final prediction = average of all trees. Hyperparameters: n_estimators=200, max_depth=15, min_samples_leaf=3. Selected as best for: Apartment, Villa, Studio, Other.

Gradient Boosting (Ensemble Boosting)

200 sequential trees each correcting the residuals of the previous. Hyperparameters: n_estimators=200, max_depth=5, learning_rate=0.1, subsample=0.8. Selected as best for: Town House, Twin House, Duplex, Penthouse.

7. Model Evaluation

7.1 Per-Type Performance

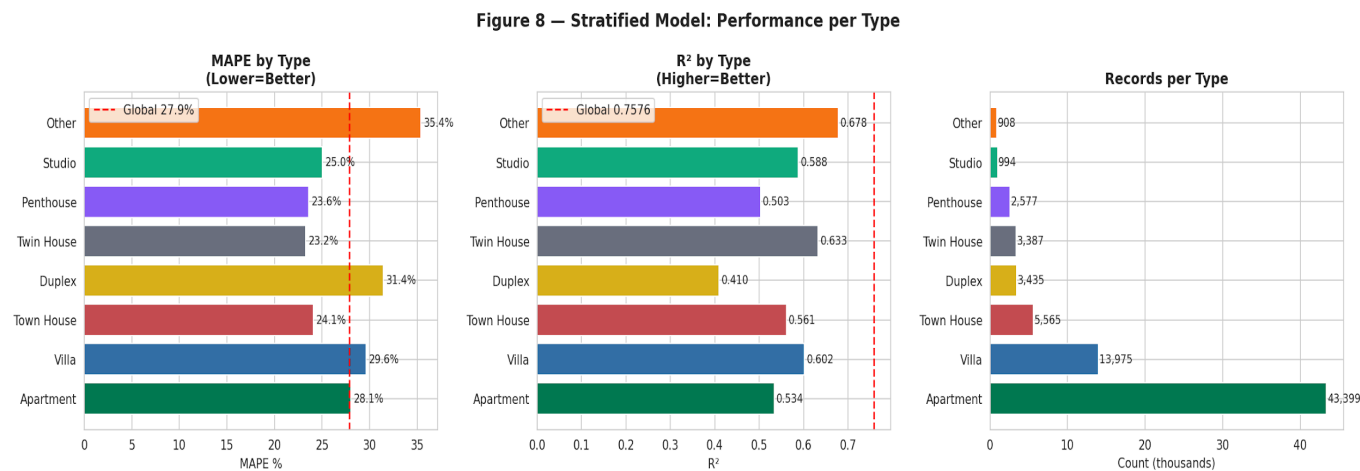


Figure 8 — MAPE, R^2 , and record count for each property type.

7.2 Actual vs. Predicted

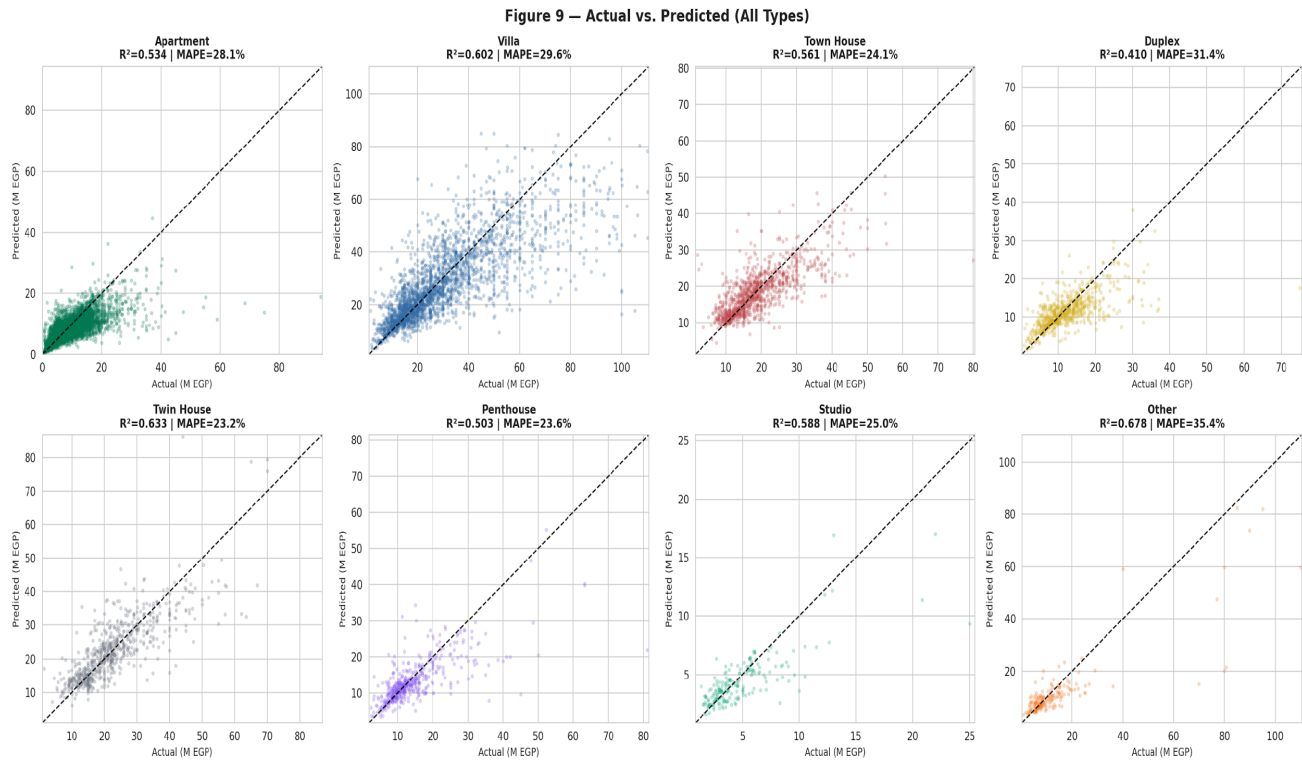


Figure 9 — Actual vs. predicted scatter for all 8 types.

7.3 Feature Importances

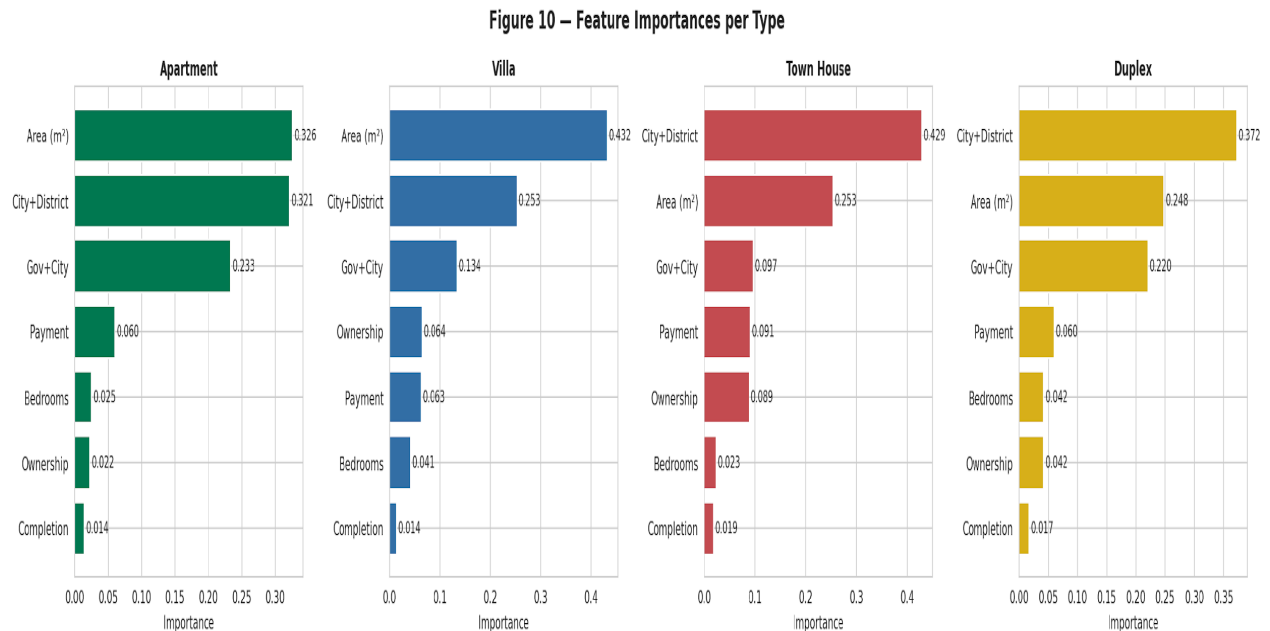


Figure 10 — Feature importances for top 4 types. City+District dominates.

8. Deployment Architecture

8.1 Technology Stack

- Python 3.11, aiohttp (scraper), Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn
- Streamlit — web app with cascading Governorate → City → District dropdowns
- Joblib — model_final.pkl (8 models + encoders)
- GitHub — source code hosting
- Streamlit Community Cloud — live public deployment

8.2 Local Setup

```
pip install -r requirements.txt
python train.py                # retrain → model_final.pkl
streamlit run app.py           # launch at http://localhost:8501
```

8.3 Streamlit Community Cloud Deployment

To deploy the app publicly on Streamlit Community Cloud (share.streamlit.io):

- Step 1: Push all code files to GitHub (exclude model_final.pkl — too large for git).
- Step 2: Go to share.streamlit.io → Sign in with GitHub → "New app".
- Step 3: Select repository, branch: main, main file: app.py → Deploy.
- Step 4: After deployment, go to App Settings → Files → upload model_final.pkl.
- Step 5: App is live at https://your-app-name.streamlit.app

Streamlit App — Main Interface:

① Property Type

Select the property type:

☒ Apartment ☐ Villa ☐ Town House ☐ Duplex ☐ Twin House ☐ Penthouse

☐ Studio ☐ Other

R ²	MAE	MAPE	Trained on
0.5336	2.06M EGP	28.1%	43,399 li...

② Location

📍 Governorate

🏠 City

📍 District / Compound

③ Apartment Details

📏 Area (m ²)	150 - +	🔑 Ownership	Primary v
🛏 Bedrooms	3 - +	💰 Payment Option	Cash v
		🏠 Completion Status	Ready v

Streamlit App — Prediction Result:

✓ Estimation Complete

Estimated Price

1.87M EGP

In Egyptian Pounds

1,873,803 EGP

Property Summary

Feature	Value
Type	Apartment
Governorate	Alexandria
City	Abu Qir
District	Khaled Ibn Al Walid St.
Area	150 m ²
Bedrooms	3
Ownership	Primary
Payment	Cash
Completion	Ready

 Apartment model (Random Forest) | $R^2 = 0.5336$ | MAE = 2.056M EGP | MAPE = 28.05% | Trained on 43,399 listings

8.4 Project Files for GitHub

- app.py — Streamlit web application
- train.py — full training pipeline
- dubizzle_scraper.py — Stage 2 aiohttp scraper (reads URLs.xlsx → writes dubizzle_listings.xlsx)
- Egypt_RE_Appraiser_Final.ipynb — complete EDA + models notebook
- requirements.txt — app dependencies (pandas, sklearn, streamlit, ...)
- requirements_scraper.txt — scraper dependencies (aiohttp, lxml, bs4, openpyxl)
- README.md — project overview, setup guide, results table
- .gitignore — excludes model_final.pkl,.xlsx, csv (too large for git)

9. Conclusions & Future Work

9.1 Key Findings

- $R^2=0.7576$, MAE=3.532M EGP, MAPE=27.87% — 28-point MAPE improvement over Kaggle baseline.
- City+District (1,473 bounded combinations) is the single most important feature.
- Ownership, Completion Status, and Payment added $\sim +0.07 R^2$ vs data.xlsx.
- Stratified models are logically correct: Town House MAPE=24.1% vs Duplex MAPE=31.4%.
- aiohttp scraper is highly efficient: 107K URLs in 4–6 hours, 99.3% success rate.
- Log-transform essential: skewness 2.63 → 0.08, MAPE reduced ~ 30 points .

9.2 Future Work

- XGBoost/LightGBM with Optuna systematic hyperparameter search — expected $+0.03$ – $0.05 R^2$.
- Floor number and finishing quality — each estimated to add $+0.05$ – $0.10 R^2$.
- Geographic coordinates (lat/lon) for spatial regression capturing micro-neighbourhood effects.
- Monthly automated re-scrape to track price trends and add temporal features.
- Confidence intervals in Streamlit app to communicate prediction uncertainty to users.